

PROYECTO FINAL DE CIBERSEGURIDAD

Análisis y Corrección de Vulnerabilidades en Debian Server

João Paulo Medeiros

Curso de Ciberseguridad · 4Geeks Academy

Entorno de Laboratorio Controlado | Red: 192.168.1.0/24

Contenido de la Presentación

01

Introducción y entorno de laboratorio

02

Metodología de pentesting

03

Vulnerabilidad 1 — FTP con acceso anónimo

04

Vulnerabilidad 2 — SSH login remoto como root

05

Vulnerabilidad 3 — WordPress con localhost

06

Vulnerabilidad 4 — Permisos inseguros wp-config.php

07

Resumen de vulnerabilidades y correcciones

08

Proceso de corrección y validación

09

Medidas preventivas y conclusión

Introducción y Entorno de Laboratorio

Red local 192.168.1.0/24 — entorno controlado

OBJETIVO DEL PROYECTO

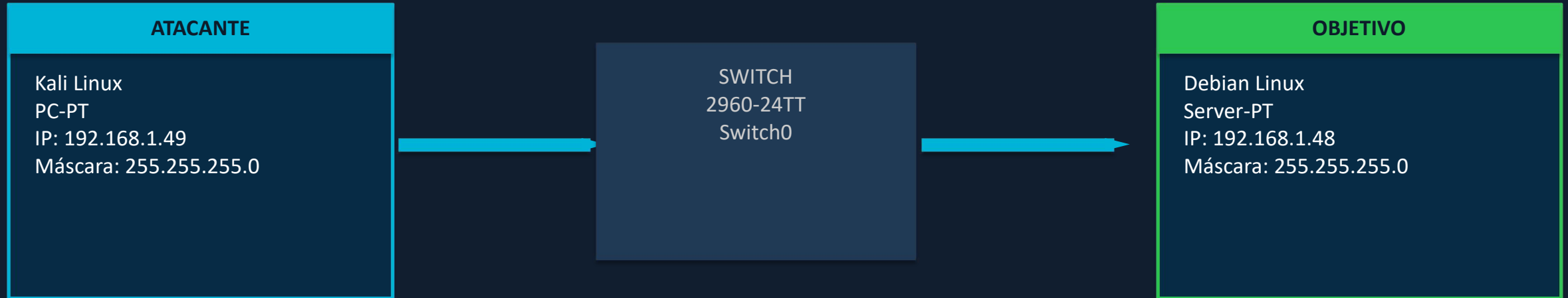
- ▶ Identificar vulnerabilidades en un servidor Debian Linux
- ▶ Evaluar el impacto potencial de cada falla detectada
- ▶ Aplicar acciones correctivas documentadas
- ▶ Validar las correcciones mediante pruebas de retesting
- ▶ Establecer medidas preventivas para reducir riesgos futuros

INFORMACIÓN DEL ENTORNO

Sistema objetivo:	Debian Linux
IP objetivo:	192.168.1.48
Máquina atacante:	Kali Linux
IP atacante:	192.168.1.49
Red:	192.168.1.0/24
Tipo de incidente:	Configuración insegura

Topología de Red

Entorno de laboratorio — red local 192.168.1.0/24



SERVICIOS EXPUESTOS EN EL SERVIDOR

FTP

Puerto 21

SSH

Puerto 22

HTTP/Apache

Puerto 80

WordPress

CMS

MariaDB

Puerto 3306

Metodología de Pentesting

Ciclo completo: reconocimiento → explotación → corrección → retesting



HERRAMIENTAS UTILIZADAS				
Nmap	FTP client	SSH client	Gobuster	MariaDB CLI
Escaneo de puertos y OS	Validación acceso anónimo	Validación login remoto	Enumeración web	Revisión config WordPress

Vulnerabilidad 1 — FTP: Acceso Anónimo Habilitado

⚠ MEDIA

VULN-01 · Severidad: MEDIA

DESCRIPCIÓN

- ▶ El servicio FTP permitía autenticación con el usuario anonymous
- ▶ No se requerían credenciales válidas para acceder
- ▶ Incrementa la superficie de ataque del servidor
- ▶ Permite enumeración de directorios y archivos
- ▶ Riesgo de exposición si los permisos son inseguros

Impacto identificado:

- ▶ Acceso no autorizado al servicio FTP
- ▶ Enumeración de directorios del servidor

EVIDENCIA

Comando ejecutado:

```
ftp 192.168.1.48
```

Credenciales utilizadas:

```
Usuario:  anonymous
Contraseña: anonymous
```

Resultado del servidor:

```
230 Login successful.
```

Verificación de permisos de escritura:

```
mkdir teste → 550 Permission denied
```

Vulnerabilidad 1 — Corrección y Validación FTP

Archivo: /etc/vsftpd.conf

ANTES — Configuración Vulnerable

```
anonymous_enable=YES
```

Estado del servicio:

- ▶ Autenticación anónima habilitada por defecto
- ▶ Cualquier usuario podía acceder sin credenciales
- ▶ Directorio raíz FTP expuesto y enumerable

DESPUÉS — Corrección Aplicada

Archivo modificado:

```
/etc/vsftpd.conf
```

Configuración aplicada:

```
anonymous_enable=NO
```

Servicio reiniciado:

```
systemctl restart vsftpd
```

Resultado de validación:

```
530 Login incorrect.  ✓ Bloqueado
```

Vulnerabilidad 2 — SSH: Login Remoto como Root

✖ CRÍTICA

VULN-02 · Severidad: CRÍTICA

DESCRIPCIÓN E IMPACTO

- ▶ El servidor permitía login SSH directo con el usuario root
- ▶ Root tiene acceso completo al sistema
- ▶ Con contraseña débil, se obtuvo acceso completo al servidor
- ▶ Posibilidad de modificar archivos del sistema
- ▶ Creación o eliminación de usuarios
- ▶ Riesgo de modificaciones no autorizadas
- ▶ Posible eliminación de logs y evidencias

EVIDENCIA

Comando ejecutado:

```
ssh root@192.168.1.48
```

Acceso obtenido:

```
root@debian:~#
```

Verificación whoami:

```
root
```

Verificación config SSH:

```
PermitRootLogin yes
```

```
cat /etc/ssh/sshd_config | grep -i permitrootlogin
```


Vulnerabilidad 2 — Corrección y Validación SSH

Archivo: /etc/ssh/sshd_config

ANTES — Configuración Vulnerable

```
PermitRootLogin yes
```

- ▶ Root login permitido por defecto
- ▶ Sin restricción de usuarios privilegiados remotos
- ▶ Credenciales débiles configuradas en laboratorio
- ▶ Acceso administrativo total desde red local

DESPUÉS — Corrección Aplicada

Archivo modificado:

```
/etc/ssh/sshd_config
```

Configuración aplicada:

```
PermitRootLogin no
```

Servicio reiniciado:

```
systemctl restart ssh
```

Resultado de validación:

```
Permission denied.  ✓ Root bloqueado
```

Vulnerabilidad 3 — WordPress: Configuración con Localhost

⚠ MEDIA

VULN-03 · Severidad: MEDIA

DESCRIPCIÓN E IMPACTO

- ▶ WordPress configurado con localhost como URL del sitio
- ▶ Redireccionamiento inválido al acceder por IP remota
- ▶ Fallo de acceso remoto al panel de administración
- ▶ Indisponibilidad parcial de la aplicación web
- ▶ Dificultad de administración remota del CMS
- ▶ Comportamiento incorrecto del sitio web en red

Herramienta de enumeración web:

```
gobuster dir -u http://192.168.1.48  
-w /usr/share/wordlists/dirb/common.txt
```

EVIDENCIA — Base de Datos

Consulta SQL ejecutada:

```
SELECT option_name, option_value  
FROM wp_options  
WHERE option_name IN ('siteurl', 'home');
```

Resultado obtenido:

```
home      | http://localhost  
siteurl   | http://localhost
```

Directorios descubiertos (Gobuster):

```
/wp-admin  /wp-login.php  /wp-content  /wp-includes
```

Vulnerabilidad 3 — Corrección y Validación WordPress

Corrección directa en la base de datos MariaDB

ANTES — Configuración Incorrecta

```
home      | http://localhost
siteurl   | http://localhost
```

- ▶ WordPress redirigía a localhost al acceder por IP
- ▶ Panel de administración inaccesible remotamente
- ▶ Comportamiento incorrecto del sitio web

DESPUÉS — Corrección Aplicada

Consultas SQL aplicadas:

```
UPDATE wp_options
SET option_value='http://192.168.1.48'
WHERE option_name='siteurl';
```

```
UPDATE wp_options
SET option_value='http://192.168.1.48'
WHERE option_name='home';
```

Resultado verificado:

```
home      | http://192.168.1.48
siteurl   | http://192.168.1.48
```

Acceso validado en navegador:

```
http://192.168.1.48/wp-login.php  ✓ OK
```

Vulnerabilidad 4 — WordPress: Permisos Inseguros en wp-config.php

⚠ MEDIA

VULN-04 · Severidad: MEDIA

DESCRIPCIÓN E IMPACTO

- ▶ El archivo wp-config.php tenía permisos 777 (-rwxrwxrwx)
- ▶ Lectura, escritura y ejecución para TODOS los usuarios del sistema
- ▶ wp-config.php contiene las credenciales de acceso a MariaDB
- ▶ Exposición de credenciales de la base de datos WordPress
- ▶ Cualquier usuario del sistema podía leer o modificar el archivo
- ▶ Riesgo de compromiso total de la base de datos

Nota:

Requiere acceso local para ser explotada, pero expone credenciales críticas.

EVIDENCIA

Comandos ejecutados:

```
find /var/www/html -name "wp-config.php"
ls -l /var/www/html/wp-config.php
```

Permisos encontrados:

```
-rwxrwxrwx (777)
```

Significado de los permisos 777:

```
rwx → Propietario: leer + escribir + ejecutar
rwx → Grupo:      leer + escribir + ejecutar
rwx → Otros:      leer + escribir + ejecutar
```

Vulnerabilidad 4 — Corrección y Validación wp-config.php

Archivo: /var/www/html/wp-config.php

ANTES — Configuración Vulnerable

```
-rwxrwxrwx (permisos 777)
```

- ▶ Lectura/escritura/ejecución para todos los usuarios
- ▶ Credenciales de MariaDB completamente expuestas
- ▶ Cualquier proceso del sistema podía leer wp-config.php
- ▶ Riesgo de compromiso de la base de datos WordPress

DESPUÉS — Corrección Aplicada

Comando aplicado:

```
sudo chmod 600 /var/www/html/wp-config.php
```

Permisos resultantes:

```
-rw----- (permisos 600)
```

Significado de los permisos 600:

```
rw-  → Propietario: solo leer + escribir
---  → Grupo:      sin acceso
---  → Otros:      sin acceso
```

Resultado de validación:

```
-rw----- ✓ Acceso restringido al propietario
```

Resumen de Vulnerabilidades Encontradas

4 vulnerabilidades identificadas, explotadas y corregidas

ID	Vulnerabilidad	Servicio	Severidad	Estado
VULN-01	FTP con acceso anónimo habilitado	FTP	MEDIA	✓ Corregida
VULN-02	Login root vía SSH permitido	SSH	CRÍTICA	✓ Corregida
VULN-03	WordPress configurado con localhost	WordPress	MEDIA	✓ Corregida
VULN-04	Permisos inseguros en wp-config.php (777)	WordPress	MEDIA	✓ Corregida

VULNERABILIDADES CRÍTICAS

1

SSH root login

VULNERABILIDADES MEDIAS

3

FTP · WordPress · wp-config.php

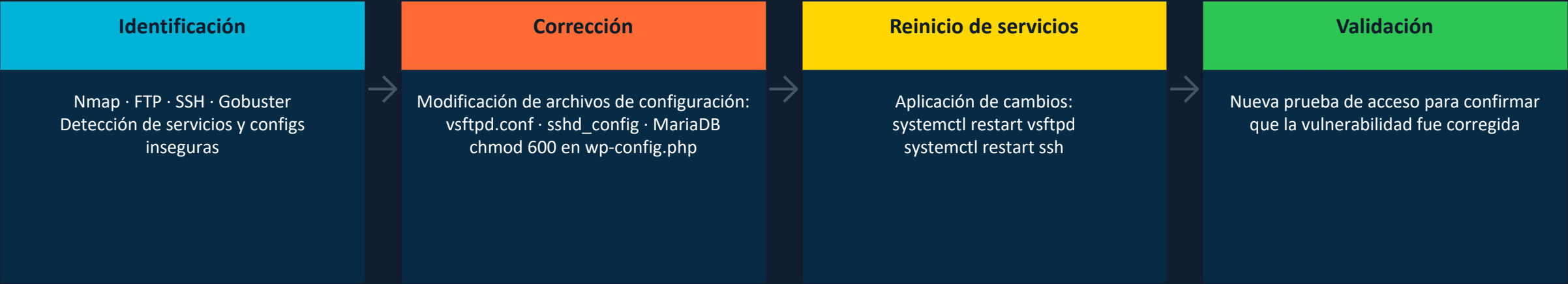
TODAS CORREGIDAS

4/4

Retesting confirmado

Proceso de Corrección y Validación

Pasos seguidos para identificar, corregir y validar cada vulnerabilidad



RESUMEN DE CORRECCIONES APLICADAS			
Vulnerabilidad	Archivo/Recurso	Cambio aplicado	Validación
FTP anónimo	/etc/vsftpd.conf	anonymous_enable=NO	530 Login incorrect
SSH root login	/etc/ssh/sshd_config	PermitRootLogin no	Permission denied
WordPress localhost	MariaDB · wp_options	siteurl/home → IP real	Panel accesible por IP
wp-config.php 777	/var/www/html/wp-config.php	chmod 600	-rw----- confirmado

Medidas Preventivas Recomendadas

Basadas en el análisis de vulnerabilidades realizado

Gestión de Accesos

- ▶ Deshabilitar auth anónima en FTP
- ▶ Deshabilitar login root por SSH
- ▶ Usar auth SSH por clave pública
- ▶ Aplicar políticas de contraseñas robustas
- ▶ Implementar MFA donde sea posible

Monitoreo y Auditoría

- ▶ Monitorear logs de autenticación
- ▶ Revisar servicios expuestos periódicamente
- ▶ Revisar logs periódicamente (journalctl)
- ▶ Realizar auditorías periódicas
- ▶ Revisar configuraciones por defecto

Hardening del Sistema

- ▶ Aplicar hardening básico en Linux
- ▶ Mantener actualizaciones del sistema
- ▶ Minimizar exposición de servicios
- ▶ Revisar configs de aplicaciones web
- ▶ Copias de seguridad periódicas

Regla general: revisar siempre las configuraciones por defecto de los servicios antes de ponerlos en producción.

Lecciones Aprendidas

Puntos clave identificados durante el análisis de seguridad

Revisar configs por defecto

FTP y SSH venían con configuraciones inseguras activadas por defecto. Siempre revisar antes de usar.

No permitir SSH como root

Con acceso root por SSH y una contraseña débil, obtuve acceso completo al servidor. Hay que desactivarlo siempre.

Verificar apps web tras instalar

WordPress apuntaba a localhost y no funcionaba en red. Hay que verificar la URL base después de instalar.

Revisar permisos de archivos

wp-config.php tenía permisos 777. Archivos con credenciales deben tener permisos más restrictivos (600).

Probar después de corregir

No basta con cambiar la config. Hay que volver a probar para confirmar que la corrección realmente funcionó.

Revisar logs del sistema

Los logs muestran intentos de acceso y sesiones abiertas. Revisarlos ayuda a detectar accesos no autorizados.

CONCLUSIÓN

- 1 Se identificaron y explotaron 4 vulnerabilidades en el servidor Debian Linux en entorno de laboratorio controlado.
- 2 La vulnerabilidad crítica (SSH root login) permitía acceso administrativo total al servidor.
- 3 Todas las vulnerabilidades fueron corregidas exitosamente y validadas mediante pruebas de retesting.
- 4 El proyecto demostró la importancia del ciclo completo: análisis → corrección → validación.

João Paulo Medeiros · Proyecto Final de Ciberseguridad · 4Geeks Academy

4 Vulnerabilidades Identificadas · 4 Correcciones Aplicadas · 4 Validaciones Confirmadas